C++ DAS Listener Implementation

Reference : RA 478

Welcome to the documentation for **RA_0478 DAS Listener in C++**.
This reference architecture demonstrates how to build a Data Analytics Solution (DAS) listener in C++ that receives messages from a test program and processes them in real-time.

General Technical Info

Component	Version
Language	C++
Standard	C++17 or later
Build System	CMake 3.14+
Environment	Windows, Linux
IGXL	> 10.30.10
MST	> 2016C
IDE	Visual Studio / Visual Studio Code / CLion

Key Features

- Real-time HTTP message reception
- Event-driven architecture with callbacks for connection, disconnection, errors, and new messages
- Structured message model including cell ID, message name, URL, payload, and timestamp
- Thread-safe implementation using C++17 features
- Message subscription filtering

- Cross-platform compatibility (Windows, Linux)
- Minimal dependencies

Architecture Overview

The DAS acts as a lightweight HTTP server that receives messages from a test cell. Each message is parsed, validated, and dispatched through an event callback system. This architecture enables adaptive test flows such as **limit adjustment**, **test enable/disable**, and **bin updates** to be handled asynchronously.

The implementation uses modern C++ practices including:

- RAII (Resource Acquisition Is Initialization) for automatic resource management
- Smart pointers for memory safety
- Lambda functions for flexible event handling
- PImpl idiom for implementation hiding and ABI stability

Example: Getting Started

Here's a quick example of how to instantiate and use the `DASMessageListener` class:

```
#include "DASMessageListener.h"
#include <iostream>

using namespace Teradyne::Archimedes::DAS;

int main() {
    std::cout << "Hello, Welcome to my C++ DAS!" << std::endl;

    // Create the listener with your DAS URL
    DASMessageListener listener("http://localhost:3000/tems/");

    // Register event callbacks
    listener.onConnected([]() {
        std::cout << "Connected to DAS." << std::endl;
    });

    listener.onDisconnected([]() {
        std::cout << "Disconnected." << std::endl;
    });

    listener.onError([](const std::string& error) {
        std::cerr << "Error: " << error << std::endl;
    });
```

```
    listener.onNewMessage([](const std::string& cellId, const AMPMessage& message) {
        std::cout << "New message from " << cellId
                  << ": " << message.messageName << std::endl;
        std::cout << "Payload: " << message.payload << std::endl;
    });

    // Optional: Subscribe to specific message types
    listener.subscribeToMessages({"TEST_START", "TEST_END", "TEST_DATA"});

    // Start the listener
    if (listener.start()) {
        std::cout << "Listener is running..." << std::endl;
        // Keep the application running
        while (listener.isRunning()) {
            std::this_thread::sleep_for(std::chrono::seconds(1));
        }
    }

    return 0;
}
```

Building the Project

Prerequisites

- C++ compiler with C++17 support (GCC 7+, Clang 5+, MSVC 2017+)
- CMake 3.14 or later

Build Instructions

```
# Navigate to the source directory
cd source/DASListener

# Create build directory
mkdir build && cd build

# Configure with CMake
cmake ..

# Build the project
cmake --build .
```

```
# Run the example application
./DASListenerExample
```

For detailed build instructions and manual compilation options, see the README file included in the source download.

Message Structure

The `AMPMessage` struct contains all relevant information about a received message:

Field	Type	Description
`cellId`	`std::string`	Test cell identifier
`messageName`	`std::string`	AMP message type (e.g., TEST_START, TEST_END)
`url`	`std::string`	Full URL of the HTTP request
`payload`	`std::string`	JSON payload containing test data
`timestamp`	`std::string`	ISO 8601 formatted timestamp

Event Callbacks

The DAS listener supports four types of event callbacks:

1. **Connected** - Triggered when the listener successfully starts
2. **Disconnected** - Triggered when the listener stops
3. **Error** - Triggered when an error occurs during operation
4. **NewMessage** - Triggered when a new AMP message is received

All callbacks are optional and can be registered using the `on*` methods.

Advanced Usage

Message Filtering

You can filter which messages to process by subscribing to specific message names:

```
std::vector<std::string> messages = {
    "TEST_START",
    "TEST_END",
```

```
    "TEST_DATA",
    "BIN_UPDATE"
};
listener.subscribeToMessages(messages);
```

If no subscriptions are set, all messages will be processed.

Thread Safety

The DAS listener implementation is thread-safe. Event callbacks are invoked from the listener's internal thread, so be mindful of synchronization if you access shared data from callbacks.

Integration with Production Systems

This implementation provides a foundation for DAS listener functionality. For production deployments, consider:

- Integrating with established HTTP libraries like cpp-httplib, Boost.Beast, or POCO
- Adding SSL/TLS support for secure communication (see RA 474 for HTTPS guidance)
- Implementing robust error recovery and logging
- Adding metrics and monitoring capabilities
- Implementing graceful shutdown mechanisms

Resources

- Download Source Code
- Build Instructions (included in source download)

Related Reference Architectures

- RA 468: C# DAS Listener - .NET implementation
- RA 473: PowerShell DAS Listener - PowerShell implementation
- RA 466: Python DAS Listener - Python implementation
- RA 474: HTTPS Management - Secure communications

This documentation is part of the Teradyne Reference Architecture series.

Source Code Files

Download Individual Files

File	Path	Description
CMakeLists.txt	root	Build configuration file
README.md	root	Library documentation (see source download)
DASMessageListener.h	include	Main header file with API definitions
DASMessageListener.cpp	src	Implementation of the DAS listener
main.cpp	examples	Example application

More details about those source code here

Quick Start

Option 1: Build with CMake

```
cd source/DASListener
mkdir build && cd build
cmake ..
cmake --build .
./DASListenerExample
```

Option 2: Manual Compilation

```
cd source/DASListener

# Compile the library
```

```
g++ -std=c++17 -c src/DASMessageListener.cpp -I include -o DASMessageListener.o

# Create static library
ar rcs libDASListener.a DASMessageListener.o

# Compile example
g++ -std=c++17 examples/main.cpp -I include -L. -lDASListener -pthread -o DASExample

# Run
./DASExample
```

Project Structure

```
DASListener/
├── CMakeLists.txt              # Build configuration
├── README.md                   # Documentation
├── include/
│   └── DASMessageListener.h    # Public API
├── src/
│   └── DASMessageListener.cpp  # Implementation
└── examples/
    └── main.cpp                # Example usage
```

Requirements

- **C++ Compiler**: GCC 7+, Clang 5+, or MSVC 2017+
- **CMake**: Version 3.14 or later (for CMake build)
- **C++ Standard**: C++17 or later
- **Platform**: Windows or Linux

Integration into Your Project

As a Static Library

1. Copy the `include/` and `src/` directories to your project
2. Add to your CMakeLists.txt:

```
add_library(DASListener STATIC
    path/to/src/DASMessageListener.cpp
)
target_include_directories(DASListener PUBLIC path/to/include)
target_link_libraries(YourApp PRIVATE DASListener Threads::Threads)
```

Header-Only Integration

You can combine the header and implementation files for header-only usage if preferred.

← Back to Overview